arXiv:2506.03030v1 [cs.LO] 3 Jun 2025

The Vampire Diary

Filip Bártek[1], Ahmed Bhayat[4], Robin Coutelier[3], Márton Hajdu[3], Matthias Hetzenberger[3], Petra Hozzová[1], Laura Kovács[3(✉)], Jakob Rath[3], Michael Rawson[5(✉)], Giles Reger[4], Martin Suda[1(✉)], Johannes Schoisswohl[3], and Andrei Voronkov[2,4(✉)]

[1] Czech Technical University in Prague, Czech Republic
martin.suda@cvut.cz
[2] EasyChair
[3] TU Wien, Vienna, Austria
laura.kovacs@tuwien.ac.at
[4] University of Manchester, UK
andrei@voronkov.com
[5] University of Southampton, UK
michael@rawsons.uk





Abstract. During the past decade of continuous development, the theorem prover Vampire has become an automated solver for the combined theories of commonly-used data structures. Vampire now supports arithmetic, induction, and higher-order logic. These advances have been made to meet the demands of software verification, enabling Vampire to effectively complement SAT/SMT solvers and aid proof assistants. We explain how best to use Vampire in practice and review the main changes Vampire has undergone since its last tool presentation, focusing on the engineering principles and design choices we made during this process.

1 Introduction

Automated reasoning has become indispensable for certifying the correctness of software systems and services [55], from Boolean satisfiability (SAT) through satisfiability modulo theories (SMT) to automated theorem proving (ATP) in first-order and higher-order logic. This tool paper describes major developments in saturation-based theorem proving, bringing our Vampire system to bear on modern software certification. Vampire now reasons efficiently in a polymorphic first-order logic with theories, induction and quantifiers, which is realized through (i) combining satisfiability solving with first-order theorem proving using the AVATAR framework [68,47]; (ii) native support for quantified reasoning with mixed arithmetic using extensions of superposition with quantifier elimination [37,56]; and (iii) embedding second-order induction schemata as inference rules in proof search [22,40]. Furthermore, (iv) Vampire has evolved to support higher-order logic [9], program synthesis [27], and finding counterexamples [49].

Our advances in saturation-based reasoning *proved to make a difference*. Vampire outperforms or complements many other state-of-the-art reasoners,

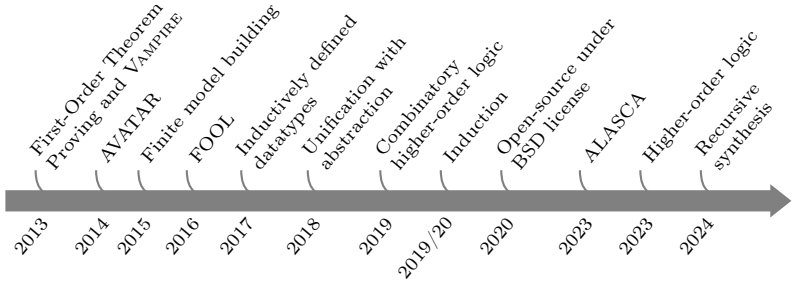


Fig. 1. Vampire timeline since our 2013 tutorial tool paper [41].

including leading SMT solvers and inductive theorem provers. For example, in CASC-J12, the most recent world championship in theorem proving, Vampire proved more problems than any other system in every competition division [64]. We believe that the increasing demand on efficient reasoning with quantifiers, theories and induction turns Vampire into a powerful solver in the automation of mathematics [17], verification of logic programs [42], ensuring system security [32], and many other areas.

This paper details the aforementioned advances in Vampire. It aims to explain how to use Vampire (Section 2) and to give an overview of the reasoning techniques used under the hood in order to allow for reasoning in more expressive logics (Section 4) and more efficient reasoning in general (Section 5). With these new features, a new permissive license, and unprecedented performance, we believe that a tool demonstration after more than a decade of continuous development is overdue. Our diary of improvements since our 2013 tutorial paper demonstration [41] is summarized in Figure 1. The present paper serves as a self-contained tool demonstration, describing the many new features Vampire supports and users can exploit. Our paper contains typical usage guidance, properly instructing readers/users interested in Vampire. For details on Vampire's calculus, inferences processes and proof search algorithms we refer to [41].

2 User Guide

Vampire ingests an input problem in either the TPTP [63] or SMT-LIB 2 [3] formats. It then attempts to show unsatisfiability[6] of the input by deriving falsum through application of its logical calculus. If it succeeds, Vampire halts and prints a step-by-step refutation. In some cases Vampire can also show satisfiability. This happens either by finding a finite model (Section 5.2) or by *saturation*: detecting that a refutation cannot be derived with a complete calculus.

[6] TPTP allows the user to supply a `conjecture`: this formula is automatically negated.

Licensing. Vampire is now open-source and available online[7] under a 3-clause BSD license. This has completely changed the Vampire team dynamics and coding culture: the development team grew, code became fully shared, and external contributors can improve Vampire. The new license has increased the user base of Vampire in teaching, research and development.

Vampire uses external code for specific tasks, including the MiniSat [18] and CaDiCaL [11] SAT solvers, the Z3 SMT solver [16] (optional), the VIRAS quantifier elimination routine [56], and `mini-gmp` [20] for arbitrary-precision arithmetic.

Installation. Vampire is written in C++17 [30] and uses the CMake [35] build system. Since the previous tool demonstration over a decade ago [41], a series of patches have improved portability and Vampire now runs on a variety of modern architectures and UNIX-like operating systems. We provide pre-compiled binaries for UNIX-like systems, and it is generally straightforward to compile Vampire from source on such systems. Support for other operating systems is more experimental, but users report success with compatibility layers such as the Windows Subsystem for Linux [43], Cygwin [45], or Cosmopolitan Libc [67].

Quick Start. It is possible, but *usually undesirable*, to invoke Vampire directly on an input file. This will cause it to run a single proof attempt, which will likely not be well-suited to the input. Instead, users should *schedule* an execution of a *portfolio* of many different *strategies*, which is achieved with the invocation

```
vampire --mode portfolio --schedule <schedule> --cores 0 <problem>
```

where `<problem>` is the input problem, `--cores 0` instructs Vampire to use all available CPU cores, and `<schedule>` should be selected from Vampire's list of built-in schedules (Section 5.4), depending on the input.

Understanding the Output. Vampire prints status messages as new strategies are launched or old strategies fail, until either a strategy succeeds or the time limit is reached. When a strategy succeeds, by default Vampire reports an SZS status [65] and then, assuming that the input is unsatisfiable, a human-readable proof. The most common SZS statuses Vampire reports are:

`Satisfiable:` input does not contain a conjecture and is satisfiable
`Unsatisfiable:` input does not contain a conjecture and is unsatisfiable
`CounterSatisfiable:` conjecture present, after negation the input is satisfiable
`Theorem:` conjecture present, after negation the input is unsatisfiable
`ContradictoryAxioms:` conjecture present, but axioms alone are unsatisfiable

Controlling Output. Vampire by default uses the SZS standards for output and reports a compact, human-readable proof. The Vampire output can be changed with the *output mode* (`-om`) and *proof format* (`-p`) flags. For example, `-om smtcomp` produces a very terse output suitable for SMT-COMP [4], while `-p tptp` produces machine-readable TSTP [66] proofs. Some more exotic formats such as LaTeX (`--latex_output`) or Dedukti [1] are under development.

[7] https://github.com/vprover/vampire/

Setting Resource Limits. Vampire can be configured to limit the amount of time (`-t <seconds>`), memory (`-m <MB>`), and on recent Linux systems the number of userspace instructions retired (`-i <MI>`[8]). The value `0` means no limit.

Exploring Options. Vampire has many, many options. A complete list can be generated with `--show_options on`, and a particular flag can be examined with `--explain`, e.g. `--explain output_mode`. Most options only affect a single strategy's behavior, but some affect the global behavior of Vampire, such as the proof format or the time limit. Single strategy options are not usually controlled by the user but automatically set by portfolio schedules (more on this in Section 5.4).

2.1 Looking Under the Hood

It is sometimes useful for users to inspect the internal state of Vampire, such as when debugging or optimizing an encoding. Here we sketch the internal mechanisms of Vampire and explain how to inspect them during operation.

Vampire works in two phases. First, the input is parsed, the conjecture — if present — is negated, and the resulting formulas are converted to clause normal form (CNF) and preprocessed. Then Vampire tries to derive the empty clause (witnessing contradiction) in order to refute the CNF of the input problem. New clauses are derived from old by applying generating inferences in Vampire's proof calculus, *superposition* [41]. The search space is partitioned into three sets of clauses: *new* clauses have been freshly derived; *passive* clauses survived Vampire's simplification efforts but have not yet participated in inference, and *active* clauses have themselves participated in inferences generating new clauses. Vampire allows inspection of these processes. To show the clauses resulting from an input, use

```
vampire --mode clausify <problem>
```

which causes Vampire to stop after preprocessing `<problem>` and print the resulting CNF. The CNF may be surprising at times as Vampire's preprocessing will happily eliminate parts of the input that it can detect will not help reaching a refutation. To inspect the progress of the preprocessing pipeline users may `--show_preprocessing on`. Once proof search begins, `--show_new on` displays new clauses (analogously, `show_passive` and `show_active`). Not all new clauses will make it to *passive*: use `--show_reductions on` to see the simplifications that Vampire applies.

3 Demonstration: Arithmetic and Induction

Consider the proposition "the sum of two lists of real numbers is equal to the sum of their concatenation". While clearly true, a formal proof requires reasoning about arithmetic, algebraic datatypes, recursive functions, polymorphism, and at

[8] Millions of instructions [61]. Instruction limits tend to more stable than time limits across hardware and operating system conditions.

least one inductive step. When given the natural first-order formalization of the problem (Fig. 2) encoded into version 2.7 of SMT-LIB, Vampire is able to find a proof immediately. We show a distilled version of the proof in mathematical notation in Figure 3.

$$\dfrac{\begin{array}{rr} & \mathrm{sum}(\epsilon) = 0 \\ \forall x : \mathbb{R}.\ \forall xs : [\mathbb{R}]. & \mathrm{sum}(x \# xs) = x + \mathrm{sum}(xs) \\ \Lambda\alpha.\ \forall ys : [\alpha]. & \epsilon +\!\!+ ys = ys \\ \Lambda\alpha.\ \forall x : \alpha.\ \forall xs, ys : [\alpha]. & (x \# xs) +\!\!+ ys = x \# (xs +\!\!+ ys) \end{array}}{\forall xs, ys : [\mathbb{R}].\ \mathrm{sum}(xs) + \mathrm{sum}(ys) = \mathrm{sum}(xs +\!\!+ ys)}$$

Fig. 2. Motivating example in polymorphic first-order logic with uninterpreted functions $\mathrm{sum} : [\mathbb{R}] \to \mathbb{R}$, $+\!\!+ : \Lambda\alpha.\ [\alpha] \times [\alpha] \to [\alpha]$, algebraic datatypes and real arithmetic.

The proof displays some new features of Vampire, in particular the use of structural induction (Sect. 4.2) and superposition-based arithmetic reasoning via the ALASCA calculus (Sect. 4.1). These features are key to Vampire's success on this problem. The need for our unique blend of induction, arithmetic and polymorphism is supported by the fact that other solvers such as CVC5 [2] or Z3 [16] cannot, to the best of our knowledge, yet process or prove problems such as this one.

4 New Capabilities

Here we present the most significant new capabilities implemented in Vampire since 2013 [41]. Improvements to existing capabilities are in Section 5.

4.1 Arithmetic Reasoning

Reasoning about arithmetic in the presence of quantifiers is highly desirable. To this end, Vampire implements the Abstracting Linear Arithmetic Superposition Calculus (Alasca) [37] that combines ideas like inequality chaining, unification with abstraction [51], and rewriting modulo linear arithmetic. Equality reasoning in Alasca can be seen as applying the superposition calculus modulo the axioms of linear arithmetic. For example, take step 18 of Figure 3. There, the literal $0 = x + \mathrm{sum}(y) - \mathrm{sum}(x \# y)$ is used to perform a rewrite $\mathrm{sum}(x \# y) \rightsquigarrow x + \mathrm{sum}(y)$ instead of a rewrite $x + \mathrm{sum}(y) - \mathrm{sum}(x \# y) \rightsquigarrow 0$, which would be the only permissible rewrite in standard superposition. In addition, Alasca uses inequality chaining and dedicated factoring rules to deal with inequalities, and combines variable elimination rules with unification with abstraction to efficiently perform unification, modulo linear arithmetic.

Non-Linear Reasoning. Alasca itself supports reasoning in linear real arithmetic with uninterpreted functions and quantifiers. Nonlinear problems are also

1. $0 = \mathrm{sum}(\epsilon)$	assumption 1 (cnf)
2. $0 = x + \mathrm{sum}(y) - \mathrm{sum}(x \# y)$	assumption 2 (cnf)
3. $\epsilon +\!\!+ x = x$	assumption 3 (cnf)
4. $y \# (z +\!\!+ x) = (y \# z) +\!\!+ x$	assumption 4 (cnf)
5. $0 \neq \mathrm{sum}(\sigma_1) + \mathrm{sum}(\sigma_0) - \mathrm{sum}(\sigma_1 +\!\!+ \sigma_0)$	conjecture (cnf)
6. $0 \neq \mathrm{sum}(\sigma_3 \# \sigma_4) + \mathrm{sum}(\sigma_0) - \mathrm{sum}((\sigma_3 \# \sigma_4) +\!\!+ \sigma_0)$ $\vee\ 0 \neq \mathrm{sum}(\epsilon) + \mathrm{sum}(\sigma_0) - \mathrm{sum}(\epsilon +\!\!+ \sigma_0)$	structural induction 5
7. $0 = \mathrm{sum}(\sigma_4) + \mathrm{sum}(\sigma_0) - \mathrm{sum}(\sigma_4 +\!\!+ \sigma_0)$ $\vee\ 0 \neq \mathrm{sum}(\epsilon) + \mathrm{sum}(\sigma_0) - \mathrm{sum}(\epsilon +\!\!+ \sigma_0)$	structural induction 5
8. $0 \neq \mathrm{sum}(\sigma_0) - \mathrm{sum}(\sigma_3 \# (\sigma_4 +\!\!+ \sigma_0)) + \mathrm{sum}(\sigma_3 \# \sigma_4)$ $\vee\ 0 \neq \mathrm{sum}(\sigma_0) + \mathrm{sum}(\epsilon) - \mathrm{sum}(\epsilon +\!\!+ \sigma_0)$	forward demodulation 6,4
9. $0 \neq \mathrm{sum}(\sigma_0) + \mathrm{sum}(\epsilon) - \mathrm{sum}(\sigma_0)$ $\vee\ 0 = \mathrm{sum}(\sigma_0) - \mathrm{sum}(\sigma_4 +\!\!+ \sigma_0) + \mathrm{sum}(\sigma_4)$	forward demodulation 7,3
10. $0 \neq \mathrm{sum}(\epsilon)$ $\vee\ 0 = \mathrm{sum}(\sigma_0) - \mathrm{sum}(\sigma_4 +\!\!+ \sigma_0) + \mathrm{sum}(\sigma_4)$	Alasca normalization 9
11. $0 \neq \mathrm{sum}(\sigma_0) + \mathrm{sum}(\epsilon) - \mathrm{sum}(\sigma_0)$ $\vee\ 0 \neq \mathrm{sum}(\sigma_0) + \mathrm{sum}(\sigma_3 \# \sigma_4) - \mathrm{sum}(\sigma_3 \# (\sigma_4 +\!\!+ \sigma_0))$	forward demodulation 8,3
12. $0 \neq \mathrm{sum}(\epsilon)$ $\vee\ 0 \neq \mathrm{sum}(\sigma_0) + \mathrm{sum}(\sigma_3 \# \sigma_4) - \mathrm{sum}(\sigma_3 \# (\sigma_4 +\!\!+ \sigma_0))$	Alasca normalization 11
13. $0 = \mathrm{sum}(\sigma_0) - \mathrm{sum}(\sigma_4 +\!\!+ \sigma_0) + \mathrm{sum}(\sigma_4)$	subsumption resolution 10,1
14. $0 \neq \mathrm{sum}(\sigma_0) + \mathrm{sum}(\sigma_3 \# \sigma_4) - \mathrm{sum}(\sigma_3 \# (\sigma_4 +\!\!+ \sigma_0))$	subsumption resolution 12,1
15. $0 \neq \mathrm{sum}(\sigma_0) + \mathrm{sum}(\sigma_3 \# \sigma_4) - (\sigma_3 + \mathrm{sum}(\sigma_4 +\!\!+ \sigma_0))$	Alasca superposition 2,14
16. $0 \neq \mathrm{sum}(\sigma_0) - \sigma_3 + \mathrm{sum}(\sigma_3 \# \sigma_4) - (\mathrm{sum}(\sigma_0) + \mathrm{sum}(\sigma_4))$	Alasca superposition 13,15
17. $0 \neq \sigma_3 - \mathrm{sum}(\sigma_3 \# \sigma_4) + \mathrm{sum}(\sigma_4)$	Alasca normalization 16
18. $0 \neq \sigma_3 - (\sigma_3 + \mathrm{sum}(\sigma_4)) + \mathrm{sum}(\sigma_4)$	Alasca superposition 2,17
19. $\square$	Alasca normalization 18

Fig. 3. Vampire's proof output of the problem from Fig. 2 in mathematical notation. The symbols σ_i are fresh Skolem constants. The induction steps are detailed in Sect. 4.2.

supported by treating nonlinear multiplications as uninterpreted functions, automatically adding the relevant axiomatization.

Mixed Integer-Real Arithmetic. While the original Alasca work is limited to real arithmetic, our current implementation in Vampire lifts these restrictions. We support reasoning in mixed integer-real arithmetic, using a tailored quantifier-elimination procedure [56], as well as various new inference rules[9] to handle the combination of mixed arithmetic and uninterpreted functions by natively supporting the rounding (floor) function.

Simplifications and Generalizations. In addition to Alasca, Vampire also provides lightweight arithmetic reasoning [48]. This includes arithmetic subterm generalization rules that complement Alasca reasoning, and other simplification rules entailed by Alasca. Although these simplification rules are not as widely applicable as Alasca, they provide for more lightweight and therefore efficient arithmetic reasoning, sufficient for many practical problems. The generalization rules include transformations like turning $\forall x, y : \mathbb{R}.\ P(3x+y)$ into the equivalent clause $\forall x : \mathbb{R}.\ P(x)$.

Integrating SMT Solvers. Vampire sometimes hands off *ground*, that is quantifier-free, arithmetic reasoning to the Z3 SMT solver [16]. This is done either by invoking AVATAR modulo theories [47] (see Sect. 5.2) or by *theory instantiation* [52]. Theory instantiation uses an SMT solver to find possible instantiations of clauses based on their purely arithmetical literals. To illustrate, consider the clause $\forall x : \mathbb{Z}.\ P(x) \vee 0 > 3x \vee x \geq 1$. The SMT solver is queried for a model satisfying $\neg(0 > 3x \vee x \geq 1)$, which is only the case for $x = 0$. The clause is instantiated with $\{x \mapsto 0\}$ and simplified to $P(0)$. Such integration of SMT solvers enables using state-of-the-art developments in SMT and is particularly beneficial for problem areas such as non-linear reasoning, for which Vampire does not yet have dedicated calculi.

4.2 Inductive Reasoning

Vampire supports inductive reasoning [40] over literals with up to one free variable.[10] It applies induction by generating theory lemmas, triggered by deriving an eligible induction goal. Vampire supports structural induction over inductively-defined datatypes [53], induction over bounded intervals of integers [29], and well-founded induction principles generated from recursive function definitions [26]. Immediately after Vampire generates the induction lemmas, it uses them to resolve their corresponding goals.

A distinctive feature of Vampire is that it seamlessly interleaves induction with other inferences, efficiently handling hundreds of thousands of induction formulas. This makes it possible to use more explosive lemma generation techniques essential for solving some inductive problems. To synthesize lemmas, Vampire

[9] The work on this inference system has not been published yet.

[10] This covers the cases of a universally-quantified conjecture, and a conjecture with any number of universally-quantified variables and one existentially-quantified variable.

can generalize over terms and term occurrences [23] or over multiple literals and clauses [24], use function definitions [26] and perform general rewriting [25].

Let us highlight some key steps of the automated induction in Vampire using the proof from Figure 3. First, when Vampire sees clause 5, it detects that induction might be in order, as clause 5 corresponds to a universally-quantified goal using an inductively-defined datatype. Therefore, Vampire uses clause 5 to instantiate the structural induction axiom for lists,

$$L[\epsilon] \wedge \forall x : \alpha, y : [\alpha].(L[y] \rightarrow L[x \# y]) \rightarrow \forall z : [\alpha].L[z],$$

by setting $\alpha := \mathbb{R}$ and $L[t] := 0 = \mathrm{sum}(t) + \mathrm{sum}(\sigma_0) - \mathrm{sum}(t +\!\!+ \sigma_0)$. Note that $L[\sigma_1]$ is set to be complementary to the literal from clause 5.

The instantiated axiom concludes that $L[z]$ holds for any $z : [\mathbb{R}]$, while clause 5 expresses that $L[\sigma_1]$ does not hold. To use this, Vampire converts the axiom into CNF, obtaining clauses $\neg L[\epsilon] \vee L[\sigma_4] \vee L[z]$ and $\neg L[\epsilon] \vee \neg L[\sigma_3 \# \sigma_4] \vee L[z]$, where σ_3, σ_4 are Skolem constants corresponding to x and y, respectively. These clauses together express that either the antecedent of the axiom does not hold (the base case $L[\epsilon]$ does not hold, or for some σ_3, σ_4 we have $L[\sigma_4]$ but not $L[\sigma_3 \# \sigma_4]$), or the conclusion that $L[z]$ is true for any z must hold. Then Vampire applies binary resolution on these two clauses with clause 5, resolving away $L[z]$, and deriving clauses 6 and 7. Clauses 6 and 7 are exactly $\neg L[\epsilon] \vee L[\sigma_4]$ and $\neg L[\epsilon] \vee \neg L[\sigma_3 \# \sigma_4]$, spelled out in full in Figure 3. The rest of the proof then covers the refutation of these two clauses.

4.3 Polymorphic Logic

Vampire now supports rank-1 polymorphic types [8] in the tradition of Standard ML [44]. This represents a trade-off between expressivity and ease of implementation. Note that Vampire does *not* presently implement a sort inference routine and all sorts in non-variable terms must be explicitly given as sort arguments [12], which may themselves be variables. For example, the axiom $\mathrm{sum}(\epsilon) = 0$ is actually represented as `sum(nil($real)) = 0`, and $\epsilon +\!\!+ ys = ys$ as `concat(A, nil(A), Ys) = Ys`. The original motivation for introducing polymorphic logic was supporting combinatory higher-order logic [7], but it has also proved useful for supporting polymorphic theories such as arrays, and for verifying programs that use parametric polymorphism.

4.4 Beyond First-Order Logic

Vampire supports extensions of first-order logic useful for software analysis and verification. In particular, Vampire implements Fool [38], a conservative extension of many-sorted first-order logic with *if-then-else* and *let-in* expressions, which can be used to capture the next-state relation of loop-free programs [39]. As such, Vampire also supports first-class *Boolean sorts*, by encoding the axiom $\forall x : o.\ x = 0 \vee x = 1$ as a new inference rule. The rule exploits the two-element domain property of the Boolean sort without blowing up proof search.

Higher-Order Logic. In addition, a branch of Vampire[11] implements a superposition-based calculus tailored for higher-order logic [10], while still using the general saturation framework from first-order logic. As higher-order unification is undecidable, our implementation bypasses eager unification by performing bounded-depth unification and introducing constraints for remaining unification terms. This technique of constraint introduction has also been used in pure first-order reasoning as delayed unification [9] and in arithmetic reasoning as unification with abstraction [37,6].

4.5 Synthesis

We further utilize Vampire's powerful proving capabilities to extend it to a program synthesizer [28,27]. Vampire works with a relational input-output specification expressed in first-order logic, capturing "for all inputs x there exists an output y such that a given relation between x and y holds". In parallel to proving this conjecture, Vampire constructs a program which computes the value of y for any given value of x. To switch on synthesis mode, use `-qa synthesis`.[12]

5 Making It Work

Taking the above extensions into account, Vampire must now prove theorems in a substantially richer logic with a much greater number of possible inferences. We now describe improvements to Vampire's core that we consider most important for meeting this challenge and maintaining good performance in practice. This adds to the observations of our previous tutorial paper [41], which remain valid. We hope to provide useful information here for those readers who develop their own reasoning systems.

5.1 Preprocessing

Computing normal forms and preprocessing remain of vital importance: the right normal form can eliminate much search space or drastically shorten the required proof. To this end Vampire has grown a new top-down clausal normal form routine [50], lifted the *blocked clause elimination* technique [34] from SAT [31], and adapted a highly effective goal-oriented rewriting technique from Twee [58]. As a general rule of thumb, preprocessing techniques have linear-time complexity, and avoid recursion to prevent stack overflow on large inputs.

5.2 Integrating SAT and SMT

One of the tricks for efficiently tackling real-life problems in rich formalisms such as first-order logic with theories is to look for sub-problems in simpler logics

[11] https://github.com/vprover/vampire/tree/hol

[12] While synthesis of recursion-free programs is available in the mainline Vampire, synthesis of recursive programs is currently in the branch `synthesis-recursive`.

and offload them to dedicated tools. In this spirit, Vampire implements the AVATAR architecture for clause splitting [68], which allows a prover to delegate the "propositional essence" of the given problem to a SAT solver. In AVATAR modulo theories [47], Vampire uses a finer abstraction[13] and delegates ground *theory* sub-problems to an SMT solver.

SAT solving is also applied within Vampire to find counterexamples to false conjectures. Vampire now provides a MACE-style finite model building mode, using a translation to SAT [14,49]. This is often a useful complement to theorem-proving modes (provided that a small counter-model exists), which helps terminate futile searches early and delivers useful insights in the form of bug traces.

5.3 Redundancy and Proof Search

Redundancy elimination is key to efficient proof search. Intuitively, a clause is redundant if it is a logical consequence of smaller clauses from the search space: checking whether a first-order clause is redundant is therefore undecidable in general. Vampire implements cheap conditions for detecting some cases of redundancy. The central technique used to implement these checks efficiently is *term indexing* [46] and here in particular *substitution trees* [19] and *code trees* [54]. Code trees are used for rewriting clauses by unit equalities [26,25] and eliminating duplicate clauses, while substitution trees are used for other inferences. To efficiently solve term ordering constraints in redundancy elimination, Vampire uses *term ordering diagrams*, which offer runtime-specialized implementations of simplification orderings [21]. Finally, Vampire also uses SAT solving to check some redundancy conditions that can be modeled as at-most-one ground constraints over the Boolean structure of clause sets [15].

5.4 A Sea of Options, Strategies, and Schedules

By *strategy* we mean a particular configuration of Vampire's option values. Since the behavior of Vampire is controlled by more than 200 options, the number of available strategies is vast. Although expert users may sometimes have an idea of which options could be well suited to tackle a problem, the prover's behavior tends to be so chaotic [61] that even expert hunches often fail. For this reason, Vampire provides *schedules* of pre-selected strategies executed sequentially, possibly adapting to the given problem's features.

Creating powerful schedules is a challenging problem. Since 2010, Vampire has employed a dedicated support tool *Spider* [69] to construct schedules from a set of training problems. Spider trials random strategies to solve as many training problems as possible and eventually selects those strategies that *complement* each other particularly well and lead to a schedule with good coverage and a short overall runtime. Techniques have recently been developed to encourage the generalization of the constructed schedule to unseen problems [5].

[13] We remark that quantifiers are always handled natively by Vampire.

Actively maintained schedules include `casc` and `casc_sat`, for general first-order theorem proving and disproving, resp., referring to the famous championship [62]. Similarly, `smtcomp` has its origin in another competition [71] and is optimized to work well on problems requiring theory reasoning. The higher-order branch (Section 4.4) provides schedules for reasoning in higher-order logic and the *Sledgehammer* [17] use-case. Finally, there is also `induction` and more.[14]

5.5 Branches

Some extensions to Vampire would have violent and extensive impact on the code base. This is true of Vampire's higher-order logic (HOL), for example. Integrating the HOL extension into Vampire would be a significant amount of work and impose a burden on all Vampire developers: but we would like it to continue, as it is a world-leading system for higher-order logic. The way we are currently dealing with this tension is by keeping this kind of feature on `git` branches, which are periodically synchronised with mainline Vampire. When a branch is widely-used enough, stable, and has a clear path to be integrated cleanly with mainline Vampire, we may consider merging it: this has happened in the past with rank-1 polymorphism and a previous approach to HOL [7].

6 Related Work and Conclusion

We have explained how best to use Vampire, discussed new features of Vampire that better align saturation-based first-order theorem proving with software verification, and described engineering required to make it work in practice.

As a first-order theorem prover with support for theories, induction and higher-order logic, Vampire has been influenced by, competes with, and might be variously compared to: SMT solvers such as `cvc5` [2] or Z3 [16]; first-order ATPs such as E [57], Spass [72], iProver [36], or Twee [58]; inductive theorem provers such as ACL2 [33], HipSpec [13], or Zeno [59]; and higher-order ATPs such as Zipperposition [70] or Leo-III [60]. Vampire distinguishes itself with its native support for quantifiers combined with calculus extensions to reason about theories, induction and higher-order logic, all tied together by highly-efficient adaptive data structures and algorithms. Naturally, Vampire integrates SAT and SMT solving for ground reasoning tasks.

This paper overviewed the main reasoning engines and practices Vampire offers in order to assist users in understanding the many ways Vampire can be integrated in other technologies. The system is under continuous development, with new applications towards proof checking and extracting system code from formal proofs. Further advances in creating tailored Vampire proof schedules for proof assistants, for example in Isabelle's *Sledgehammer* [17], are also under active development.

[14] Use `--explain_option schedule` to list schedules available from your Vampire.

Acknowledgements. We would like to thank all users and prior developers who contributed to Vampire. We acknowledge the valuable Vampire contributions made by Daneshvar Amrollahi, Ioan Dragan, Bernhard Gleiss, Bernhard Kragl, Kryštof Hoder, Evgenii Kotelnikov, Alexandre Riazanov, Martin Riener, Simon Robillard, Boris Shminke, and Eva Maria Wagner.

This research was funded in whole or in part by the ERC Consolidator Grant ARTIST 101002685, the ERC Proof of Concept Grant LEARN 101213411, the TU Wien Doctoral College SecInt, the FWF SpyCoDe Grant 10.55776/F85, the WWTF grant [ForSmart Grant ID: 10.47379/ICT22007], and the Amazon Research Award 2023 QuAT. Martin Suda was supported by the project CORESENSE no. 101070254 under the Horizon Europe programme and by the Czech Ministry of Education, Youth and Sports under the ERC CZ project POSTMAN no. LL1902. Petra Hozzová was supported by the European Union under the project ROBOPROX (reg. no. CZ.02.01.01/00/22_008/0004590).

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

1. Assaf, A., Burel, G., Cauderlier, R., Delahaye, D., Dowek, G., Dubois, C., Gilbert, F., Halmagrand, P., Hermant, O., Saillard, R.: Dedukti: a Logical Framework based on the $\lambda\Pi$-Calculus Modulo Theory. CoRR **abs/2311.07185** (2023)
2. Barbosa, H., Barrett, C.W., Brain, M., Kremer, G., Lachnitt, H., Mann, M., Mohamed, A., Mohamed, M., Niemetz, A., Nötzli, A., Ozdemir, A., Preiner, M., Reynolds, A., Sheng, Y., Tinelli, C., Zohar, Y.: cvc5: A Versatile and Industrial-Strength SMT Solver. In: TACAS. pp. 415–442 (2022)
3. Barrett, C., Fontaine, P., Tinelli, C.: The Satisfiability Modulo Theories Library (SMT-LIB). `www.SMT-LIB.org` (2016)
4. Barrett, C., de Moura, L., Stump, A.: SMT-COMP: Satisfiability modulo Theories Competition. In: CAV. p. 20–23 (2005)
5. Bártek, F., Chvalovský, K., Suda, M.: Regularization in Spider-Style Strategy Discovery and Schedule Construction. In: IJCAR. pp. 194–213 (2024)
6. Bhayat, A., Korovin, K., Kovács, L., Schoisswohl, J.: Refining Unification with Abstraction. In: LPAR. pp. 36–47 (2023)
7. Bhayat, A., Reger, G.: A Combinator-Based Superposition Calculus for Higher-Order Logic. In: IJCAR. pp. 278–296 (2020)
8. Bhayat, A., Reger, G.: A Polymorphic Vampire (Short Paper). In: IJCAR. pp. 361–368 (2020)
9. Bhayat, A., Schoisswohl, J., Rawson, M.: Superposition with Delayed Unification. In: CADE. pp. 23–40 (2023)
10. Bhayat, A., Suda, M.: A Higher-Order Vampire (Short Paper). In: IJCAR. pp. 75–85 (2024)
11. Biere, A., Faller, T., Fazekas, K., Fleury, M., Froleyks, N., Pollitt, F.: Cadical 2.0. In: CAV. pp. 133–152 (2024). https://doi.org/10.1007/978-3-031-65627-9_7, https://doi.org/10.1007/978-3-031-65627-9_7

12. Blanchette, J.C., Paskevich, A.: TFF1: The TPTP Typed First-Order Form with Rank-1 Polymorphism. In: CADE. pp. 414–420 (2013)
13. Claessen, K., Johansson, M., Rosén, D., Smallbone, N.: Automating Inductive Proofs Using Theory Exploration. In: CADE (2013)
14. Claessen, K., Sörensson, N.: New Techniques that Improve MACE-style Model Finding. In: WS on Model Computation - Principles, Algorithms and Applications (2003)
15. Coutelier, R., Rath, J., Rawson, M., Biere, A., Kovács, L.: SAT Solving for Variants of First-Order Subsumption. Formal Methods in System Design (2024)
16. De Moura, L., Bjørner, N.: Z3: An Efficient SMT Solver. In: TACAS. pp. 337–340 (2008)
17. Desharnais, M., Vukmirović, P., Blanchette, J., Wenzel, M.: Seventeen Provers Under the Hammer. In: ITP. pp. pp. 8:1–8:18 (2022)
18. Eén, N., Sörensson, N.: An Extensible SAT-solver. In: SAT. pp. 502–518 (2003)
19. Graf, P.: Substitution Tree Indexing. In: RTA. pp. 117–131 (1995)
20. Granlund, T.: The GNU Multiple Precision Arithmetic Library (2023), https://gmplib.org/gmp-man-6.3.0.pdf
21. Hajdu, M., Coutelier, R., Kovács, L., Voronkov, A.: Term Ordering Diagrams. In: CADE (2025), to appear
22. Hajdu, M., Hozzová, P., Kovács, L., Reger, G., Voronkov, A.: Getting Saturated with Induction. In: Principles of Systems Design. pp. 306–322 (2022)
23. Hajdu, M., Hozzová, P., Kovács, L., Schoisswohl, J., Voronkov, A.: Induction with Generalization in Superposition Reasoning. In: CICM. pp. 123–137 (2020)
24. Hajdu, M., Kovacs, L., Rawson, M., Voronkov, A.: The Vampire Approach to Induction. EasyChair Preprint no. 9217 (EasyChair, 2022)
25. Hajdu, M., Kovács, L., Rawson, M.: Rewriting and Inductive Reasoning. In: LPAR. pp. 278–294 (2024)
26. Hajdu, M., Hozzová, P., Kovács, L., Voronkov, A.: Induction with Recursive Definitions in Superposition. In: FMCAD. pp. 1–10 (2021)
27. Hozzová, P., Amrollahi, D., Hajdu, M., Kovács, L., Voronkov, A., Wagner, E.M.: Synthesis of Recursive Programs in Saturation. In: IJCAR. p. 154–171 (2024)
28. Hozzová, P., Kovács, L., Norman, C., Voronkov, A.: Program synthesis in saturation. In: CADE. pp. 307–324 (2023)
29. Hozzová, P., Kovács, L., Voronkov, A.: Integer Induction in Saturation. In: CADE. pp. 361–377 (2021)
30. ISO: ISO/IEC 14882:2017: Programming languages — C++. International Organization for Standardization, Geneva, Switzerland (Dec 2017)
31. Järvisalo, M., Biere, A., Heule, M.: Blocked Clause Elimination. In: TACAS. pp. 129–144 (2010)
32. Jeanteur, S., Kovács, L., Maffei, M., Rawson, M.: CryptoVampire: Automated Reasoning for the Complete Symbolic Attacker Cryptographic Model. In: SP. pp. 3165–3183 (2024)
33. Kaufmann, M., Manolios, P., Moore, J.S.: Computer-Aided Reasoning: An Approach, vol. 3. Springer (06 2000). https://doi.org/10.1007/978-1-4615-4449-4
34. Kiesl, B., Suda, M., Seidl, M., Tompits, H., Biere, A.: Blocked Clauses in First-Order Logic. In: LPAR. pp. 31–48 (2017)
35. Kitware, I.: CMake (2025), https://cmake.org/
36. Korovin, K.: iProver — An Instantiation-Based Theorem Prover for First-Order Logic (System Description). In: IJCAR. pp. 292–298 (2008)
37. Korovin, K., Kovács, L., Reger, G., Schoisswohl, J., Voronkov, A.: ALASCA: Reasoning in Quantified Linear Arithmetic. In: TACAS. pp. 647–665 (2023)

38. Kotelnikov, E., Kovács, L., Reger, G., Voronkov, A.: The Vampire and the FOOL. In: CPP. pp. 37–48 (2016)
39. Kotelnikov, E., Kovács, L., Voronkov, A.: A FOOLish Encoding of the Next State Relations of Imperative Programs. In: IJCAR. pp. 405–421 (2018)
40. Kovács, L., Hozzová, P., Hajdu, M., Voronkov, A.: Induction in Saturation. In: IJCAR. pp. 21–29 (2024)
41. Kovács, L., Voronkov, A.: First-Order Theorem Proving and Vampire. In: CAV. pp. 1–35 (2013)
42. Lifschitz, V., Lühne, P., Schaub, T.: Towards Verifying Logic Programs in the Input Language of clingo. In: Fields of Logic and Computation III. pp. 190–209 (2020)
43. Microsoft: Windows Subsystem for Linux (WSL), https://ubuntu.com/desktop/wsl
44. Milner, R.: The Definition of Standard ML: Revised. MIT press (1997)
45. Racine, J.: The Cygwin Tools: a GNU Toolkit for Windows (2000)
46. Ramakrishnan, I.V., Sekar, R., Voronkov, A.: Term Indexing. In: Handbook of Automated Reasoning, pp. 1853–1964. Elsevier and MIT Press (2001)
47. Reger, G., Bjørner, N.S., Suda, M., Voronkov, A.: AVATAR Modulo Theories. In: GCAI. pp. 39–52 (2016)
48. Reger, G., Schoisswohl, J., Voronkov, A.: Making Theory Reasoning Simpler. In: TACAS. pp. 164–180 (2021)
49. Reger, G., Suda, M., Voronkov, A.: Finding Finite Models in Multi-sorted First-Order Logic. In: SAT. pp. 323–341 (2016)
50. Reger, G., Suda, M., Voronkov, A.: New Techniques in Clausal Form Generation. In: GCAI. pp. 11–23 (2016)
51. Reger, G., Suda, M., Voronkov, A.: Unification with Abstraction and Theory Instantiation in Saturation-Based Reasoning. In: TACAS. pp. 3–22 (2018)
52. Reger, G., Suda, M., Voronkov, A.: Unification with Abstraction and Theory Instantiation in Saturation-Based Reasoning. In: TACAS. pp. 3–22 (2018)
53. Reger, G., Voronkov, A.: Induction in Saturation-Based Proof Search. In: CADE. pp. 477–494 (2019)
54. Riazanov, A., Voronkov, A.: Partially Adaptive Code Trees. In: JELIA. pp. 209–223 (2000)
55. Rungta, N.: A Billion SMT Queries a Day (Invited Paper). In: CAV. pp. 3–18 (2022)
56. Schoisswohl, J., Kovács, L., Korovin, K.: VIRAS: Conflict-Driven Quantifier Elimination for Integer-Real Arithmetic. In: LPAR. pp. 147–164 (2024)
57. Schulz, S., Cruanes, S., Vukmirović, P.: Faster, Higher, Stronger: E 2.3. In: CADE. pp. 495–507 (2019)
58. Smallbone, N.: Twee: An Equational Theorem Prover. In: CADE. pp. 602–613 (2021)
59. Sonnex, W., Drossopoulou, S., Eisenbach, S.: Zeno: An Automated Prover for Properties of Recursive Data Structures. In: TACAS. pp. 407–421 (2012)
60. Steen, A., Benzmüller, C.: The Higher-Order Prover Leo-III. In: IJCAR. pp. 108–116 (2018)
61. Suda, M.: Vampire Getting Noisy: Will Random Bits Help Conquer Chaos? (System Description). In: IJCAR. pp. 659–667 (2022)
62. Sutcliffe, G.: The CADE ATP System Competition - CASC. AI Magazine **37**(2), 99–101 (2016)
63. Sutcliffe, G.: The Logic Languages of the TPTP World. Logic Journal of the IGPL (2022). https://doi.org/10.1093/jigpal/jzac068

64. Sutcliffe, G.: The 12th IJCAR Automated Theorem Proving System Competition — CASC-J12. The European Journal on Artificial Intelligence **0**(0), 30504554241305110 (0)
65. Sutcliffe, G.: The SZS Ontologies for Automated Reasoning Software. In: LPAR Workshops (2008)
66. Sutcliffe, G.: The TPTP World — Infrastructure for Automated Reasoning. In: LPAR. pp. 1–12 (2010)
67. Tunney, J.: Cosmopolitan Libc (2025), https://justine.lol/cosmopolitan/
68. Voronkov, A.: AVATAR: The Architecture for First-Order Theorem Provers. In: CAV. pp. 696–710 (2014)
69. Voronkov, A.: Spider: Learning in the Sea of Options (2023), https://easychair.org/smart-program/Vampire23/2023-07-05.html#talk:223833
70. Vukmirovic, P., Bentkamp, A., Blanchette, J., Cruanes, S., Nummelin, V., Tourret, S.: Making Higher-Order Superposition Work. J. Autom. Reason. **66**(4), 541–564 (2022)
71. Weber, T., Conchon, S., Déharbe, D., Heizmann, M., Niemetz, A., Reger, G.: The SMT competition 2015-2018. J. Satisf. Boolean Model. Comput. **11**(1), 221–259 (2019), https://doi.org/10.3233/SAT190123
72. Weidenbach, C., Dimova, D., Fietzke, A., Kumar, R., Suda, M., Wischnewski, P.: SPASS Version 3.5. In: CADE. pp. 140–145 (2009)